Secondary School Portal

Technical Requirements Document (TRD) — Version 1.0

This document defines the architecture, directory structure, module boundaries, data models, security guidelines, and development roadmap for the Secondary School Portal. It serves as the authoritative blueprint for engineering the platform.

1. Technology Stack

- Frontend: React with TypeScript, offering a modern, responsive user experience.
- Backend API: Go (Golang) REST API, built for optimal throughput and low latency.
- Database: PostgreSQL, utilized as a relational database to ensure ACID compliance.
- Authentication: JSON Web Tokens (JWT) for secure, stateless session tracking.
- Infrastructure: Docker and Docker Compose for local development and reliable staging orchestration.
- Architecture: Modular Monolith to enforce clean domain separation while maintaining deployment simplicity.

2. Directory Structure

The codebase is organized as a clean, modular monolith structured according to Go standard layout patterns:

- cmd/main.go — Application entry point, handles dependency injection, server initialization, and routing registration.
- config/config.go — Application configuration loading from environment variables and validation routines.
- internal/auth/ — Package housing core cryptographic utilities, token validation logic, and password hashing primitives.
- internal/middleware/ — Common HTTP interceptors (CORS, Request Logging, Recovery, JWT extraction, and RBAC).
- internal/handler/ — Interface adapters translating incoming HTTP payloads to internal service inputs.
- internal/service/ — Orchestrator package handling transaction boundaries and business logic validations.
- internal/repository/ — Data access layer interacting with the PostgreSQL database.
- internal/model/ — Standard schema definitions, domain structures, and JSON mapping attributes.
- migrations/ — Plain SQL schema evolution files ordered sequentially (001_users.up.sql through 015_notifications.up.sql).
- .env — Local environment configurations (database secrets, server port, token keys).
- go.mod & go.sum — Module dependencies and checksum pinning.
- docker-compose.yml — Orchestration configuration for local development infrastructure.

3. Module Design & Architecture

To prevent code bloat, there are NO dedicated student_service.go, teacher_service.go, or admin_service.go files. The User module serves as the central entity managing accounts, role definitions, and system profiles.

Core Module	Handler Responsibilities	Repository Capabilities
Auth	Login, refresh, logout flow, token issuance.	Token blocklist, active sessions metadata.
User	Manage profiles (ADMIN, TEACHER, STUDENT), state changes.	User CRUD, profile data joins, state toggles.
Class	Classrooms, subjects, enrollments, teacher assignments.	Enrollments, subject listings, teacher mappings.
Result	Grade publishing, record edits, transcript exports.	Bulk uploads, term averaging, GPA processing.
Attendance	Daily registration, absence tracking, stats generation.	Calendar calculations, aggregate compliance query.
Fee	Fee structures, item allocation, dynamic billing rules.	Dynamic fee items, aggregate school terms balances.
Payment	Initialize payments, complete transactions, webhooks.	Idempotent tx logs, fee balance settlement.
Notification	Broadcast alerts, alert templates, mark as read.	Unread count, state updating, historical logs.

3.1 Request Execution Pipeline

Requests flow through a predictable, layered lifecycle:

1. Client: Transmits payload containing HTTP method, headers, and request body.
2. Router: Matches URI pattern and identifies targeted application controller.
3. Middleware Stack: Processes cross-cutting concerns (CORS, Logger, Recovery, Auth JWT parsing, RBAC validation).
4. Handler: Binds JSON request body, validates structural constraints, calls business logic layer.
5. Service: Orchestrates entity actions, processes business logic, starts transaction scopes.
6. Repository: Executes parameterized SQL operations against PostgreSQL database.
7. PostgreSQL: Records transaction changes or fetches current states under ACID rules.
8. Response Pipeline: Service serializes system models, Handler returns status codes and clean JSON payloads back to the Client.

4. Database Design & Migration Sequence

The PostgreSQL relational model must enforce strict cascading rules and indexes. The schema migration order is sequenced to support dependency relationships without foreign-key conflicts:

1. 001_users: Core identity tables storing password hashes, primary roles, and status states.
2. 002_academic_sessions: Year-spanning periods (e.g., '2026/2027') to group system data.
3. 003_terms: Partition of academic sessions (e.g., 'First Term', 'Second Term').
4. 004_classes: Physical or logical classrooms mapping study levels.
5. 005_subjects: Curriculum records cataloging teachable courses (e.g., 'Mathematics').
6. 006_student_enrollments: Junction mapping users to classes and academic terms.
7. 007_teacher_assignments: Mapping assigning teachers to subject blocks and classes.
8. 008_results: Academic reports container summarizing terminal student grades.
9. 009_result_scores: Individual grade sheets containing test, assessment, and exam details.
10. 010_attendance: Record logs indexing student presence on scheduled class dates.
11. 011_fee_structures: Catalog of expected cost sets tied to academic levels.
12. 012_fee_items: Specific components of a fee structure (e.g., 'Tuition fee', 'Laboratory fee').
13. 013_student_fee_accounts: Ledger records linking concrete outstanding debts to student profiles.
14. 014_payments: Audit trails tracking financial transactions made to resolve fees.
15. 015_notifications: System notices issued to communication targets.

5. Key Business Flows

5.1 Teacher Result Authorization Pattern

To prevent unauthorized grade modification, the system enforces contextual role-based security checks on result uploads:

1. JWT Analysis: Request intercepted; user identity and role are resolved. If the role is not TEACHER, the request is instantly rejected.
2. Assignment Verification: The Service queries teacher_assignments to verify if the requesting user is mapped to the target class and subject.
3. Context Check: If an assignment exists, proceeding actions (upload, edit, publish) are allowed. Otherwise, a 403 Forbidden is returned.

5.2 Secure Payment & Fee Settlement Flow

Payment handling requires tight transaction control and integration with external gateways:

1. Initialization: Payment is requested; system stores a PENDING record with a unique external transaction reference.
2. Gateway Redirection: Secure payment metadata is passed to the client to execute checkout.
3. Webhook Handshake: Gateway posts encrypted state update. The system verifies the webhook's origin signature.
4. Idempotency Guard: Checks database to confirm the reference has not been processed. If already updated, it returns immediately.
5. Atomic Transaction: The database starts a transaction, updates the payment record to SUCCESS, adjusts the student's ledger balances, and logs the change.
6. Commit & Notify: Transaction commits; notification event is emitted to inform the user.

6. Security Implementation Standards

The platform enforces strict standards to guarantee application and customer data integrity:

- Cryptography: Password data is encoded using bcrypt. Plaintext passwords are never stored.
- Session Lifespan: JWT expiration matches a strict short-term validity window to mitigate token capture risks.
- Access Controls: Authorization logic is dual-layered (static route patterns in middleware, resource ownership checks in services).
- Parameterized Queries: All database persistence layers use SQL parameters. Raw string formatting is strictly prohibited.
- Data Hygiene: Incoming parameters are parsed through strict struct validation constraints prior to domain processing.
- Secrets Management: Keys, configurations, and database credentials must only be injected via environment variables.
- CORS & Protection: Cross-Origin Resource Sharing rules restrict domains to predefined origins. Panic recoveries capture runtime errors safely without leakages.

7. Implementation Roadmap

Phase	Focus Scope	Milestone Target
Phase 1	Base environment configuration, local Docker, PostgreSQL.	Containerization and schema migrations ready.
Phase 2	User module, Auth JWT services, middleware layers.	Authentication & RBAC operational.
Phase 3	Classroom assignments, academic sessions, enrollment controls.	Functional academic structure.
Phase 4	Grade submission system, attendance tracking.	Authorized grading mechanisms.
Phase 5	Fee accounts generation, webhook handling, payments.	End-to-end ledger verification.
Phase 6	Notification engine, UI integrations, security hardening.	Production ready deployment.
Phase 7	Redis caching, MinIO attachment storage, background tasks.	Scale and performance enhancements.

8. Conclusion

This Technical Requirements Document provides a clear architecture for a highly secure, reliable, and scalable Secondary School Portal. By maintaining a clean modular monolith structure in Go, the application ensures high development velocity, operational stability, and straightforward deployment patterns.